

Optimization and algorithms

V. Leclère (ENPC)

April 3rd, 2026

Why should I bother to learn this stuff?

- Being able to recognize the type of problem is the first step toward finding the right tool to address it.
- Having an idea of the tools available to you will help choose one.
- $\implies$ useful for any engineer (or intern) that might have to model and then solve a practical optimization problem.

Contents

1 Classification of optimization problems

- Generic ideas
- Some important classes

2 Optimization methods

- Heuristics
- Descent direction method
- Model based methods

3 Wrap-up

Contents

1 Classification of optimization problems

- Generic ideas
- Some important classes

2 Optimization methods

- Heuristics
- Descent direction method
- Model based methods

3 Wrap-up

Why bother with classes of optimization problems?

Consider a function $f : X \rightarrow \mathbb{R}$, and the following optimization problem

$$\begin{array}{ll} \text{Min} & f(x) \\ x \in \mathbb{R}^n & \\ \text{s.t.} & x \in X \end{array}$$

Solving this problem can be more or less hard depending on the class in which f and $X \subset \mathbb{R}^{n1}$ belongs.

Determining in which class a problem belongs is quite important:

- some problems can be solved for n of order 10 at most, other for n of order 10^6 or more;
- the methodological approach to tackle different problems vary wildly;
- the numerical tools (e.g. solvers) also...

You must be able to (roughly) classify correctly the problem you face, in order to know what can be done or not.

¹There is also an important theory of optimization where X is not contained in a finite-dimensional space (PDE control, calculus of variations), which will not be discussed here.

Classification with respect to the objective function f



- f **linear** is the simplest case
- f **quadratic** is a very important case, simple if f is convex
- f **smooth** (e.g. $\mathcal{C}^2$) allow to use first and second order information on f
- (f polynomial is a special case, with specific algorithms)
- f **convex** implies that any local minimum is a global minimum

In a convex setting, finding global optima is tractable and certifiable. Otherwise, it is generally hard. We often settle for local optima or stationary points. There exists other specific cases where global optima can be found (e.g. some non-convex quadratic problems) but they are rare.

The algorithms we present are mainly for smooth functions. Convergence theory will be done in the convex case.

Classification with respect to the objective function f



- f **linear** is the simplest case
- f **quadratic** is a very important case, simple if f is convex
- f **smooth** (e.g. $\mathcal{C}^2$) allow to use first and second order information on f
- (f polynomial is a special case, with specific algorithms)
- f **convex** implies that any local minimum is a global minimum

In a convex setting, finding global optima is tractable and certifiable. Otherwise, it is generally hard. We often settle for local optima or stationary points. There exists other specific cases where global optima can be found (e.g. some non-convex quadratic problems) but they are rare.

The algorithms we present are mainly for smooth functions. Convergence theory will be done in the convex case.



- f **linear** is the simplest case
- f **quadratic** is a very important case, simple if f is convex
- f **smooth** (e.g. $\mathcal{C}^2$) allow to use first and second order information on f
- (f polynomial is a special case, with specific algorithms)
- f **convex** implies that any local minimum is a global minimum

In a convex setting, finding global optima is tractable and certifiable. Otherwise, it is generally hard. We often settle for local optima or stationary points. There exists other specific cases where global optima can be found (e.g. some non-convex quadratic problems) but they are rare.

The algorithms we present are mainly for smooth functions. Convergence theory will be done in the convex case.

Classification with respect to the constraint set



- $X = \mathbb{R}^n$, is known as **unconstrained optimization**.
- $X = \{x \in \mathbb{R}^n \mid Ax = b\}$, can be cast, up to reparametrization, as unconstrained optimization. It might be more efficient to directly deal with the constraints.
- $X = \{x \in \mathbb{R}^n \mid \underline{x}_i \leq x_i \leq \bar{x}_i, \quad \forall i \in [n]\}$ is the **box constrained** optimization.
- $X = \{x \in \mathbb{R}^n \mid Ax \leq b\}$ is a polyhedron.
- X convex, generally given as $\{x \in \mathbb{R}^n \mid Ax = b, \quad h_j(x) \leq 0, \quad \forall j \in [n_I]\}$ with h_j convex.
- If X is a finite set we speak of **combinatorial optimization**.
- X can also be non-convex but smooth (e.g. a manifold)

A few comments:

- Unconstrained optimization is by far easier.
- Box constraints, and sometimes spherical constraints, are easy.
- Polyhedral constraints indicate LP-based methods.
- Integer constraints make the problem a lot harder and change the nature of the optimization methods.

Classification with respect to the constraint set



- $X = \mathbb{R}^n$, is known as **unconstrained optimization**.
- $X = \{x \in \mathbb{R}^n \mid Ax = b\}$, can be cast, up to reparametrization, as unconstrained optimization. It might be more efficient to directly deal with the constraints.
- $X = \{x \in \mathbb{R}^n \mid \underline{x}_i \leq x_i \leq \bar{x}_i, \quad \forall i \in [n]\}$ is the **box constrained** optimization.
- $X = \{x \in \mathbb{R}^n \mid Ax \leq b\}$ is a polyhedron.
- X convex, generally given as $\{x \in \mathbb{R}^n \mid Ax = b, \quad h_j(x) \leq 0, \quad \forall j \in [n_I]\}$ with h_j convex.
- If X is a finite set we speak of **combinatorial optimization**.
- X can also be non-convex but smooth (e.g. a manifold)

A few comments:

- Unconstrained optimization is by far easier.
- Box constraints, and sometimes spherical constraints, are easy.
- Polyhedral constraints indicate LP-based methods.
- Integer constraints make the problem a lot harder and change the nature of the optimization methods.

Contents

1 Classification of optimization problems

- Generic ideas
- Some important classes

2 Optimization methods

- Heuristics
- Descent direction method
- Model based methods

3 Wrap-up

Least-square problem (LS)

$$\text{Min}_{x \in \mathbb{R}^n} \|Ax - b\|_2$$

- Equivalent *in the sense of minimizers* to $\text{Min}_{x \in \mathbb{R}^n} \|Ax - b\|_2^2$ (since $t \mapsto t^2$ increases on $\mathbb{R}_+$)
- $\leadsto$ convex, **not everywhere differentiable** (nondiff. when $Ax = b$); the squared form is convex **and smooth**
- Explicit solution via linear algebra (QR / SVD) in medium scale;
- Large-scale: iterative methods (LSQR / CG), exploiting sparsity / matrix-free products
- Typical scale (sparse, good structure): up to $n \sim 10^6$ – 10^8

Exercise: functional approximation

♣ Exercise: We consider a physical function Φ that is approximated as the superposition of multiple simple phenomena (e.g. waves). Each simple phenomenon $p \in [P]$ is represented by a function $\Phi_p : \mathbb{R}^d \rightarrow \mathbb{R}$.

We have data points $(x^k, y^k)_{k \in [n]}$, and want to find the Φ that match *at best* the data while being a linear combination of Φ_p .

Propose a least-square regression that answers this question.



$$\begin{array}{ll}\text{Min}_{x \in \mathbb{R}^n} & c^T x \\ \text{s.t.} & Ax = b \\ & A'x \leq b'\end{array}$$

- convex problem with linear objective and polyhedral constraint set
- a rare case where exact solution can be obtained
- easily solved through dedicated code, open-source (e.g. GLPK, HiGHS) or proprietary² (e.g. CPLEX, Gurobi)
- can be solved for $nnz \approx 10^7$ (non-zeros coefficients of A)
- very important case in practice and as a subroutine for other problems
- two main (class of) algorithms:
 - ▶ simplex algorithm (seen in 1A)
 - ▶ interior point method (discussed later in this course)

²Licences are expensive(!!) but free for students!

Quadratic optimization problem (QP)



$$\begin{array}{ll} \text{Min}_{x \in \mathbb{R}^n} & \frac{1}{2} x^\top Q x + c^\top x \\ \text{s.t.} & Ax = b \\ & A'x \leq b' \end{array}$$

- quadratic objective and polyhedral constraint set
- exact solution can be obtained
- easily solved if $Q \succeq 0$, hard otherwise
- can be solved for $nnz \approx 10^6$ (convex case, good structure)

Exercise: Lasso as QP

♣ Exercise: A classical extension of the least-square problem, which has strong theoretical and practical interest is the LASSO problem

$$\underset{x \in \mathbb{R}^p}{\text{Min}} \quad \|Ax - y\|^2 + \lambda \|x\|_1$$

Show that this problem can be cast as a QP problem.

Quadratically constrained quadratic problem (QCQP)

$$\begin{array}{ll} \text{Min}_{x \in \mathbb{R}^n} & \frac{1}{2} x^\top Q x + c^\top x \\ \text{s.t.} & \frac{1}{2} x^\top P_i x + q_i^\top x \leq b_i \quad \forall i \in [k] \\ & A x = b \\ & A' x \leq b' \end{array}$$

- Reasonably easy if convex (i.e if Q and P_i are positive semidefinite)
- can be solved for $n \approx 10^5$
- less important than previous examples

Exercise: binary optimization is equivalent to QCQP

♣ Exercise: Consider the following optimization problem.

$$\begin{array}{ll} \text{Min}_{x \in \mathbb{R}^n} & c^\top x \\ \text{s.t.} & Ax = b \\ & x_i \in \{0, 1\} \quad \forall i \in I \end{array}$$

Write this problem as a QCQP. Is it convex ?

Second order cone problem (SOCP)

$$\begin{array}{ll} \text{Min}_{x \in \mathbb{R}^n} & c^\top x \\ \text{s.t.} & \|A_i x + b_i\|_2 \leq c_i^\top x + d_i \quad \forall i \in [k] \\ & Ax = b \\ & A'x \leq b' \end{array}$$

- convex problem
- can be solved for $nnz \geq 10^6$, through most "linear" solver, relying on interior points methods
- equivalent to convex QCQP
- extend the modeling power of LP
- appears naturally in robust optimization

Exercise: robust linear programming

♠ Exercise: Consider the following **robust** linear program

$$\begin{array}{ll} \underset{x \in \mathbb{R}^n}{\text{Min}} & c^\top x \\ \text{s.t.} & (a_i + R_i \delta_i)^\top x \leq b_i \quad \forall \|\delta_i\|_2 \leq 1, \quad \forall i \in [m] \end{array}$$

Write this problem as a SOCP.



$$\begin{array}{ll} \text{Min}_{X \in S_n(\mathbb{R})} & \text{tr}(CX) \\ \text{s.t.} & A(X) = b \\ & X \succeq 0 \end{array}$$

where X , and C are symmetric matrices, and $A : S_n \rightarrow \mathbb{R}^m$ a linear mapping.

- convex problem
- can be solved for $nnz \geq 10^3$, through some "linear" solver, relying on interior points methods
- contains SOCP
- limited in size in part because the number of actual variables is n^2



$$\underset{x}{\text{Min}} \quad f(x)$$

where f is convex, finite, and differentiable.

- Iterative algorithm yields ε -solution
- Solutions are global due to convexity
- Complexity theory is well understood: maximum theoretical speed, and algorithms matching this speed
- Convergence speed is better under strong convexity assumptions
- Can be solved for $n \geq 10^4$

↪ this is where we will spend most of our time

Unconstrained convex non-differentiable optimization

$$\underset{x}{\text{Min}} \quad f(x)$$

where f is convex and finite.

- Iterative algorithm yields ε -solution
- Solutions are global due to convexity
- Complexity theory is well understood: maximum theoretical speed, and algorithms matching this speed
- Can be solved for $n \geq 10^3$

Unconstrained differentiable optimization

$$\underset{x}{\text{Min}} \quad f(x)$$

where f is differentiable.

- Iterative algorithm yields ε stationary point (i.e. $\|\nabla f(x)\| \leq \varepsilon$)
- Algorithms are mostly the same as in convex differentiable setting, but the theory is more involved
- Can find a stationary point for $n \geq 10^4$

↪ most algorithms presented in this course can be used to hopefully get to a locally optimal point.

Constrained convex optimization

$$\begin{array}{ll} \text{Min} & f(x) \\ \text{s.t.} & x \in X \end{array}$$

where X is a convex set.

- Easiest if X is a box or ball
- Specific approach relying on LP if X is a polyhedron
- Various methods in the generic case:
 - ▶ projection
 - ▶ feasible direction
 - ▶ constraint penalization
 - ▶ dualization

Combinatorial problem

$$\min_{x \in X} f(x)$$

where X is a finite set.

- This roughly represent the problem of **combinatorial optimization**.
- X being finite you can, in theory, test all possibilities and choose the best. However, this **brute force** approach is often unpractical due to the size of X .
- Even if an exact solution can be obtained, it is not often the case.
- Finding lower-bounds is interesting to understand how far your current solution is from the optimum.
- Practical methods are often *matheuristics* or *meta-heuristics* adapted to the specificity of the problem.
- Problems are often very hard, and practical solvability depends on the specific problem structure.



$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & c^T x \\ \text{s.t.} \quad & Ax = b, x \geq 0 \\ & x_i \in \mathbb{N} \quad \forall i \in I \subset [n] \end{aligned}$$

- A very important class of problem, with huge modeling power.
- By order of difficulty we distinguish: continuous variables, binary variables and integer variables.
- Exact solution methods rely on the idea of branch and cut.
(<https://www.youtube.com/watch?v=2zKCQ03Jz0Y> (13'))
- Very powerful (commercial) solvers (like Gurobi, Cplex, Mosek...) are developed and improved every year to tackle these problems. They use a mix of insightful mathematical ideas and heuristic knowledge.
- Efficiency of the solver depends on the type of problem, and the formulation of the problem.



$$\begin{array}{ll}\min_{x \in \mathbb{R}^n} & c^T x \\ \text{s.t.} & Ax = b \\ & x \geq 0 \\ & x \in C \\ & x_i \in \mathbb{N} \quad \forall i \in I \subset [n]\end{array}$$

where C is a convex cone.

- Harder than MILP
- More recent development, thus the theory and heuristic experience is less advanced than MILP
- Numerical efficiency is quickly improving

Exercise: stock optimization

♠ Exercise: Consider that you sell a given product over T days. The demand for each day is d_t . Having a quantity x_t of items in stock have a cost (per day) of cx_t . You can order, each day, a quantity q_t , and have to satisfy the demand.

For each of the following variations: model the problem, give the class to which it belongs, and give the optimal solution if easily found.

- ① Without any further constraints / specifications.
- ② There is an "ordering cost": each time you order, you have to pay a fixed cost κ .
- ③ Instead of an "ordering cost" there is a maximum number of days at which you can order a replenishment.

Solver families and typical scale rule of thumb



- **Dense second-order methods** (Newton, generic interior-point)
 - ▶ Each iteration requires factorizing an $n \times n$ matrix
 - ▶ Typical scale: $n \lesssim 10^3$ (dense), up to 10^4 with strong sparsity
 - ▶ High accuracy, few iterations
- **Sparse direct methods** (sparse Cholesky / LU inside IPM or SQP)
 - ▶ Cost dominated by fill-in and sparsity pattern
 - ▶ Typical scale: $n \sim 10^4$ – 10^6 in favorable cases
 - ▶ Common for LP, convex QP, SOCP
- **First-order methods** (gradient, proximal, ADMM, coordinate descent)
 - ▶ Each iteration uses gradients or simple proximal operators
 - ▶ Typical scale: $n \sim 10^5$ – 10^8 if operations are cheap/sparse
 - ▶ Lower accuracy, many iterations
- **Stochastic / streaming methods** (SGD and variants)
 - ▶ Use only partial data at each iteration
 - ▶ Typical scale: $n \gtrsim 10^6$, data size 10^7 + and beyond
 - ▶ Dominant in machine learning

Contents

1 Classification of optimization problems

- Generic ideas
- Some important classes

2 Optimization methods

- Heuristics
- Descent direction method
- Model based methods

3 Wrap-up

Contents

1 Classification of optimization problems

- Generic ideas
- Some important classes

2 Optimization methods

- **Heuristics**
- Descent direction method
- Model based methods

3 Wrap-up

"Ad Hoc" solution

A **heuristic** is an admissible, not necessarily optimal, solution to a given optimization problem. It gives upper-bounds.

- In a lot of applications, experience or good sense, can give reasonably good heuristics.
- Sometimes these heuristics can have a few parameters that can be tuned by trial and error.

♣ Exercise: In the stock optimization example, with fixed ordering cost, propose a simple heuristic.

♣ Exercise: Now assume that, in this same example, there is some uncertainty on the demand, adapt your heuristic to offer a *robustness* parameter.

"Ad Hoc" solution

A **heuristic** is an admissible, not necessarily optimal, solution to a given optimization problem. It gives upper-bounds.

- In a lot of applications, experience or good sense, can give reasonably good heuristics.
- Sometimes these heuristics can have a few parameters that can be tuned by trial and error.

♣ Exercise: In the stock optimization example, with fixed ordering cost, propose a simple heuristic.

♣ Exercise: Now assume that, in this same example, there is some uncertainty on the demand, adapt your heuristic to offer a *robustness* parameter.

"Ad Hoc" solution

A **heuristic** is an admissible, not necessarily optimal, solution to a given optimization problem. It gives upper-bounds.

- In a lot of applications, experience or good sense, can give reasonably good heuristics.
- Sometimes these heuristics can have a few parameters that can be tuned by trial and error.

♣ Exercise: In the stock optimization example, with fixed ordering cost, propose a simple heuristic.

♣ Exercise: Now assume that, in this same example, there is some uncertainty on the demand, adapt your heuristic to offer a *robustness* parameter.

Random search

A good way of obtaining *good* solutions is to randomly test multiple admissible solutions, and keep the best one.

Examples:

- exhaustive search (combinatorial)
- genetic algorithms
- simulated annealing
- particle swarm

Use case :

- hard problems (combinatorial or continuous) where finding an admissible solution is easy
- when you just want an admissible solution, if possible better than what you had

Random search

A good way of obtaining *good* solutions is to randomly test multiple admissible solutions, and keep the best one.

Examples:

- exhaustive search (combinatorial)
- genetic algorithms
- simulated annealing
- particle swarm

Use case :

- hard problems (combinatorial or continuous) where finding an admissible solution is easy
- when you just want an admissible solution, if possible better than what you had

Video resources

<https://www.youtube.com/watch?v=3QJjfeVrut8> (5')

<https://www.youtube.com/watch?v=NI3W11rvWoc> (4')

Contents

1 Classification of optimization problems

- Generic ideas
- Some important classes

2 Optimization methods

- Heuristics
- **Descent direction method**
- Model based methods

3 Wrap-up

Descent methods



Consider the **unconstrained** optimization problem

$$\min_{x \in \mathbb{R}^n} f(x). \quad (1)$$

A *descent direction algorithm* is an algorithm that constructs a sequence of points $(x^{(k)})_{k \in \mathbb{N}}$, that are recursively defined with:

$$x^{(k+1)} = x^{(k)} + t^{(k)} d^{(k)} \quad (2)$$

where

- $x^{(0)}$ is the initial point,
- $d^{(k)} \in \mathbb{R}^n$ is the descent direction,
- $t^{(k)}$ is the step length.

→ most of this is discussed in next classes.



Consider the **unconstrained** optimization problem

$$\min_{x \in \mathbb{R}^n} f(x). \quad (1)$$

A *descent direction algorithm* is an algorithm that constructs a sequence of points $(x^{(k)})_{k \in \mathbb{N}}$, that are recursively defined with:

$$x^{(k+1)} = x^{(k)} + t^{(k)} d^{(k)} \quad (2)$$

where

- $x^{(0)}$ is the initial point,
- $d^{(k)} \in \mathbb{R}^n$ is the descent direction,
- $t^{(k)}$ is the step length.

→ most of this is discussed in next classes.



Consider the **unconstrained** optimization problem

$$\min_{x \in \mathbb{R}^n} f(x). \quad (1)$$

A *descent direction algorithm* is an algorithm that constructs a sequence of points $(x^{(k)})_{k \in \mathbb{N}}$, that are recursively defined with:

$$x^{(k+1)} = x^{(k)} + t^{(k)} d^{(k)} \quad (2)$$

where

- $x^{(0)}$ is the initial point,
- $d^{(k)} \in \mathbb{R}^n$ is the descent direction,
- $t^{(k)}$ is the step length.

→ most of this is discussed in next classes.

Video explanation

<https://www.youtube.com/watch?v=n-Y0SDS0fUI>(5')

Descent direction

For a differentiable objective function f , $d^{(k)}$ will be a descent direction iff $\nabla f(x^{(k)}) \cdot d^{(k)} < 0$, which can be seen from a first order development:

$$f(x^{(k)} + t d^{(k)}) = f(x^{(k)}) + t \langle \nabla f(x^{(k)}), d^{(k)} \rangle + o(t).$$

The most classical descent direction is $d^{(k)} = -\nabla f(x^{(k)})$, which corresponds to the gradient algorithm.

Descent direction

For a differentiable objective function f , $d^{(k)}$ will be a descent direction iff $\nabla f(x^{(k)}) \cdot d^{(k)} < 0$, which can be seen from a first order development:

$$f(x^{(k)} + t d^{(k)}) = f(x^{(k)}) + t \langle \nabla f(x^{(k)}), d^{(k)} \rangle + o(t).$$

The most classical descent direction is $d^{(k)} = -\nabla f(x^{(k)})$, which corresponds to the gradient algorithm.

Step-size choice

The step-size $t^{(k)}$ can be:

- fixed $t^{(k)} = t^{(0)}$, for all iteration,
- optimal $t^{(k)} \in \arg \min_{t \geq 0} f(x^{(k)} + td^{(k)})$,
- a "good" step, following some rules (e.g Armijo's rules).

Finding the optimal step size is a special case of unidimensional optimization (or line search).

Step-size choice

The step-size $t^{(k)}$ can be:

- fixed $t^{(k)} = t^{(0)}$, for all iteration,
- optimal $t^{(k)} \in \arg \min_{t \geq 0} f(x^{(k)} + td^{(k)})$,
- a "good" step, following some rules (e.g Armijo's rules).

Finding the optimal step size is a special case of unidimensional optimization (or line search).

Contents

1 Classification of optimization problems

- Generic ideas
- Some important classes

2 Optimization methods

- Heuristics
- Descent direction method
- Model based methods

3 Wrap-up



Another class of algorithm consists in constructing a simple **model** of the objective function f that is optimized and then refined.

Generally speaking, model-based algorithm goes as follows:

- 1 Solve $\min_{x \in X} f^k(x)$
- 2 Update model f^k into f^{k+1}

This approach might work if

- The model problem $\min_{x \in X} f^k(x)$ is *simple*
- The model f^k locally looks like the true function f around the optimum



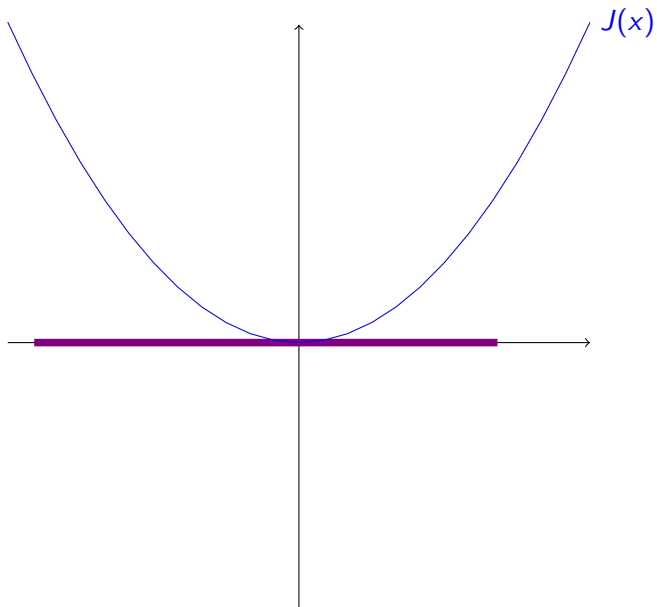
Another class of algorithm consists in constructing a simple **model** of the objective function f that is optimized and then refined.

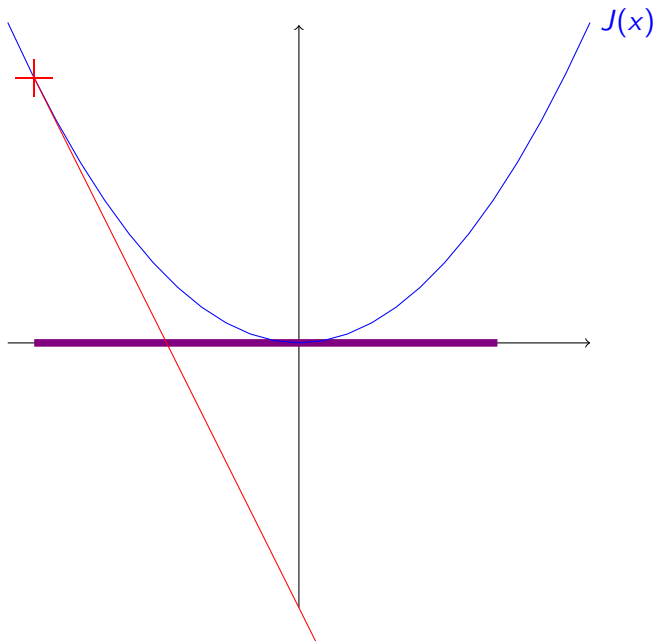
Generally speaking, model-based algorithm goes as follows:

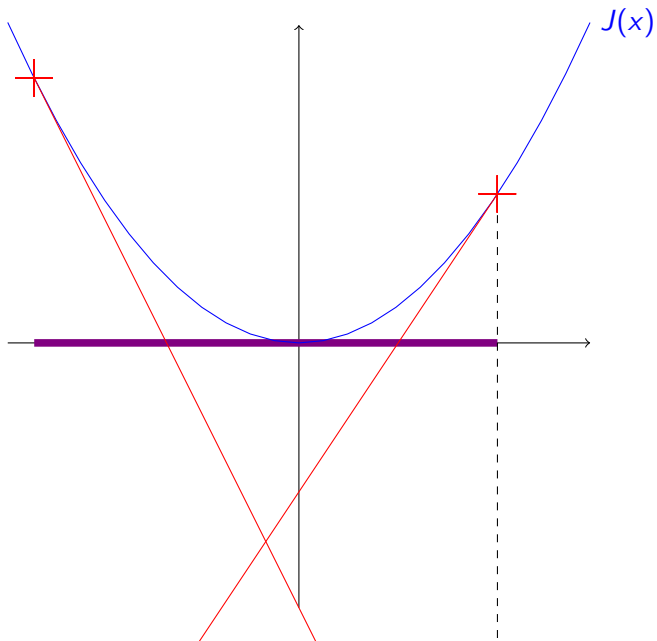
- 1 Solve $\min_{x \in X} f^k(x)$
- 2 Update model f^k into f^{k+1}

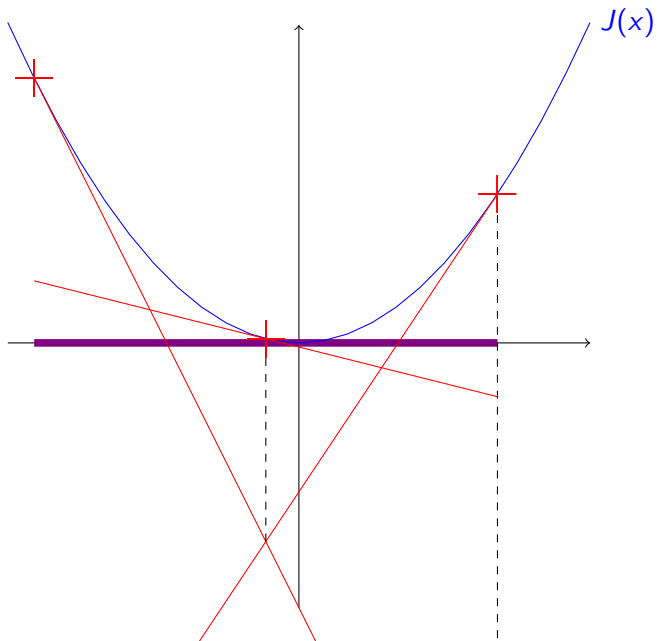
This approach might work if

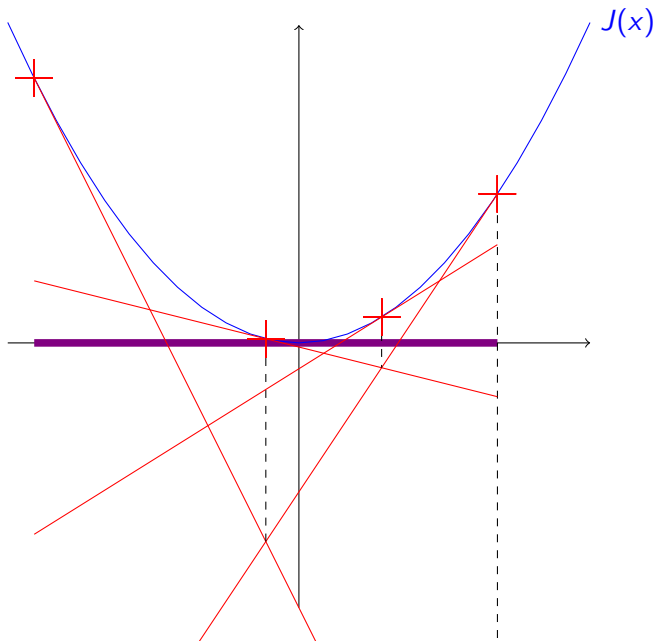
- The model problem $\min_{x \in X} f^k(x)$ is *simple*
- The model f^k locally looks like the true function f around the optimum











Kelley algorithm

Data: Convex objective function f , Compact set X , Initial point $x_0 \in X$

Result: Admissible solution $x^{(k)}$, lower-bound $\underline{v}^{(k)}$

Set $f^{(0)} \equiv -\infty$;

for $k \in \mathbb{N}$ **do**

 Compute a subgradient $g^{(k)} \in \partial f(x^{(k)})$;

 Define a cut $\mathcal{C}^{(k)} : x \mapsto f(x^{(k)}) + \langle g^{(k)}, x - x^{(k)} \rangle$;

 Update the lower approximation $f^{(k+1)} = \max\{f^{(k)}, \mathcal{C}^{(k)}\}$;

 Solve $(P^{(k)}) : \min_{x \in X} f^{(k+1)}(x)$;

 Set $\underline{v}^{(k)} = \text{val}(P^{(k)})$;

 Select $x^{(k+1)} \in \text{sol}(P^{(k)})$;

end

Algorithm 1: Kelley's cutting plane algorithm



Consider an unconstrained, non-linear, smooth problem

$$\underset{x \in \mathbb{R}^n}{\text{Min}} \quad f(x)$$

The idea of trust region is based on the following two facts:

- f locally looks like it's second order limited development

$$f(x + h) = f(x) + \langle \nabla f(x), h \rangle + \frac{1}{2} h^\top \nabla^2 f(x) h + o(\|h\|^2)$$

- we know how to compute the minimum of a quadratic function on a ball.

Trust region method



The trust region method goes as follows, given a current point x^k and *trust radius* Δ^k

- 1 compute $f^k(x^k + h) = f(x^k) + \langle \nabla f(x^k), h \rangle + \frac{1}{2} h^\top \nabla^2 f(x^k) h$
- 2 solve $\min_{y \in B(x^k, \Delta^k)} f^k(y)$, with optimal solution y^k
- 3 compute $f(y^k)$
- 4 compute the *concordance* r^k as the ratio actual decrease / model decrease

$$r^k = \frac{f(x^k) - f(y^k)}{f^k(x^k) - f^k(y^k)}$$

- ▶ If r^k is small, the model is bad and you decrease Δ^k and restart the iteration
 - ▶ If r^k is large (close to 1) update the current point $x^{k+1} = y^k$.
- 5 If r^k is close to one and y^k is on the boundary, increase Δ^k .

↪ there are full books on trust region methods.



The trust region method goes as follows, given a current point x^k and *trust radius* Δ^k

- ① compute $f^k(x^k + h) = f(x^k) + \langle \nabla f(x^k), h \rangle + \frac{1}{2} h^\top \nabla^2 f(x^k) h$
- ② solve $\min_{y \in B(x^k, \Delta^k)} f^k(y)$, with optimal solution y^k
- ③ compute $f(y^k)$
- ④ compute the *concordance* r^k as the ratio actual decrease / model decrease

$$r^k = \frac{f(x^k) - f(y^k)}{f^k(x^k) - f^k(y^k)}$$

- ▶ If r^k is small, the model is bad and you decrease Δ^k and restart the iteration
 - ▶ If r^k is large (close to 1) update the current point $x^{k+1} = y^k$.
- ⑤ If r^k is close to one and y^k is on the boundary, increase Δ^k .

↪ there are full books on trust region methods.

Video explanation

<https://www.youtube.com/watch?v=G-QKRv1rgG0>(30')

Some optimization problems in Python

<https://www.youtube.com/watch?v=sJ5HTi70wXo>(10')

Contents

1 Classification of optimization problems

- Generic ideas
- Some important classes

2 Optimization methods

- Heuristics
- Descent direction method
- Model based methods

3 Wrap-up

What you have to know

- Important elements defining an optimization problem : continuous/discrete, smooth/non-differentiable, convex/non-convex, linear/non-linear, constrained/unconstrained.
- Main optimization classes: LP, MILP, differentiable unconstrained, combinatorial.
- The difference between heuristic and exact methods
- Main classes of exact method : descent direction, approximation method.

What you really should know

- Other important classes of optimization problem (LS, QP, SOCP, SDP)
- Some ideas of heuristic methods (simulated annealing, genetic algorithms)
- Kelley's cutting plane algorithm
- Principle of trust region method

What you have to be able to do

- Recognise a LP / MILP
- Recognise a (convex) differentiable optimization problem, constrained or not

What you should be able to do

- know how to use a "lift" variable, e.g.

$$\begin{aligned} \min_x \max(f_1(x), f_2(x)) &= \min_{x,z} z \\ \text{s.t. } f_1(x) &\leq z \\ f_2(x) &\leq z \end{aligned}$$